

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO5 – Articulation	Assessment:	Assessment Tool 1 – Constraint-Based Problem Solving
Weightage:	35% of CIA		
CO5 Statement:	Implement semantic models using predicate logic, word sense disambiguation and validate information retrieval techniques. (BL-5)		
Student Name:	VIMALA K	Reg. No.:	192424276
Section / Batch:	B.Tech (AI&DS)	Date:	26/08/26

Code of Conduct

I VIMALA K 192424276 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:



Instructions

- Draw necessary diagrams such as constraint graphs, coreference chains, discourse trees, or predicate logic representations wherever applicable.
- Generate solutions that fully satisfy all given constraints and provide clear evaluation of the output.
- Answers should be concise, well-structured, and supported by evidence from the given text/scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Discourse, Reference Resolution & Text Coherence	Apply constraint-based problem solving for reference resolution, identify agreement and coherence constraints, resolve coreferences, and ensure entity chain consistency in a given paragraph.	Q1
Dialog and Conversational Agents, Discourse Planning & Surface Realizations	Construct constrained dialog responses by applying dialog acts, discourse planning, coherence and politeness constraints, generate multiple valid responses, and evaluate the best output.	Q2
Machine Translation, Discourse Structure, Predicate Logic & Interpretation	Perform word sense disambiguation using constraints, construct predicate logic representations, maintain discourse structure (rhetorical relations), and generate coherent translations/paraphrases.	Q3

Annexure A – Constraint-Based Problem Solving

1. Reference Resolution and Discourse Coherence

Given the following paragraph:

"Ravi met Arun at the library. He was looking for a book on Artificial Intelligence. Arun helped him find it. Later, they discussed the book before leaving."

Tasks:

a) Identify all the referring expressions and their possible antecedents.

b) Apply the following constraints to resolve the references:

- **Gender and number agreement**
- **Recency**
- **Grammatical role**
- **Semantic compatibility**
- **Discourse coherence**

c) Construct a table showing:

Referring Expression Possible Antecedents Valid/Invalid Reason

d) Provide the final coreference chains.

e) Rewrite the paragraph by replacing pronouns with their correct references.

f) Discuss what ambiguity would arise if the sentence "He was looking for a book" appeared before introducing Arun.

Coding:

```
entities = ["Ravi", "Arun", "book"]
```

```
references = {
```

```
    "He": ["Ravi", "Arun"],
```

```
    "it": ["book"],
```

```
    "him": ["Ravi", "Arun"],
```

```
    "they": ["Ravi", "Arun"]
```

```
}
```

```
resolved = {  
    "He": "Arun",  
    "it": "book",  
    "him": "Ravi",  
    "they": "Ravi and Arun"  
}
```

```
print("REFERRING EXPRESSIONS")
```

```
print("-" * 40)
```

```
for expression, antecedents in references.items():
```

```
    print(expression, "->", antecedents)
```

```
print("\nRESOLVED REFERENCES")
```

```
print("-" * 40)
```

```
for expression, antecedent in resolved.items():
```

```
    print(expression, "->", antecedent)
```

```
print("\nCOREFERENCE CHAINS")
```

```
print("-" * 40)
```

```
print("Ravi -> him")
```

```
print("Arun -> He")
```

```
print("book -> it -> the book")
```

```
print("Ravi + Arun -> they")
```

```
print("\nRESOLVED PARAGRAPH")
```

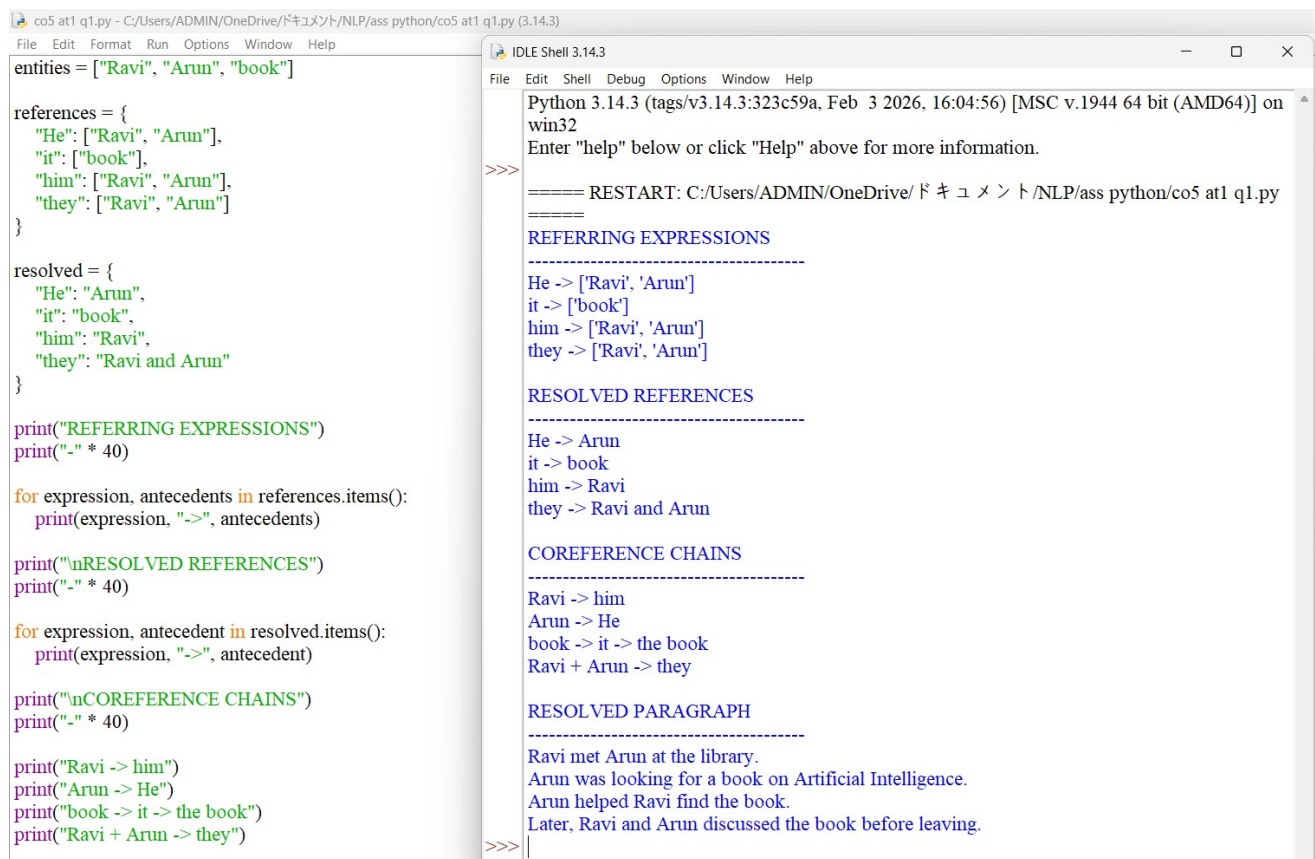
```
print("-" * 40)
```

```
print("Ravi met Arun at the library.")
```

```
print("Arun was looking for a book on Artificial Intelligence.")
```

```
print("Arun helped Ravi find the book.")
```

```
print("Later, Ravi and Arun discussed the book before leaving.")
```



The screenshot shows a Python IDE with two windows. The left window displays the source code for a coreference resolution script. The right window shows the output of the script, which includes a list of referring expressions, resolved references, coreference chains, and the final resolved paragraph.

```
co5 at1 q1.py - C:/Users/ADMIN/OneDrive/ドキュメント/NLP/ass python/co5 at1 q1.py (3.14.3)
File Edit Format Run Options Window Help
entities = ["Ravi", "Arun", "book"]

references = {
    "He": ["Ravi", "Arun"],
    "it": ["book"],
    "him": ["Ravi", "Arun"],
    "they": ["Ravi", "Arun"]
}

resolved = {
    "He": "Arun",
    "it": "book",
    "him": "Ravi",
    "they": "Ravi and Arun"
}

print("REFERRING EXPRESSIONS")
print("-" * 40)

for expression, antecedents in references.items():
    print(expression, "->", antecedents)

print("\nRESOLVED REFERENCES")
print("-" * 40)

for expression, antecedent in resolved.items():
    print(expression, "->", antecedent)

print("\nCOREFERENCE CHAINS")
print("-" * 40)

print("Ravi -> him")
print("Arun -> He")
print("book -> it -> the book")
print("Ravi + Arun -> they")

print("\nRESOLVED PARAGRAPH")
print("-" * 40)

print("Ravi met Arun at the library.")
print("Arun was looking for a book on Artificial Intelligence.")
print("Arun helped Ravi find the book.")
print("Later, Ravi and Arun discussed the book before leaving.")
```

Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/ADMIN/OneDrive/ドキュメント/NLP/ass python/co5 at1 q1.py =====
REFERRING EXPRESSIONS

He -> ['Ravi', 'Arun']
it -> ['book']
him -> ['Ravi', 'Arun']
they -> ['Ravi', 'Arun']
RESOLVED REFERENCES

He -> Arun
it -> book
him -> Ravi
they -> Ravi and Arun
COREFERENCE CHAINS

Ravi -> him
Arun -> He
book -> it -> the book
Ravi + Arun -> they
RESOLVED PARAGRAPH

Ravi met Arun at the library.
Arun was looking for a book on Artificial Intelligence.
Arun helped Ravi find the book.
Later, Ravi and Arun discussed the book before leaving.
>>>

2. Dialog Act Recognition and Response Generation

Conversation History:

Student: "I submitted my assignment, but I am worried that I may have made many mistakes."

Required Dialog Acts:

Reassure + Advise

Constraints:

- 1. Maintain entity coherence using assignment, mistakes, and you.**
- 2. Use a Cause–Effect or Elaboration discourse relation.**
- 3. Include at least two keywords from:**
 - review
 - improve
 - confident
 - feedback
- 4. Response length must be 2–3 sentences.**
- 5. Maintain a positive and polite tone.**
- 6. Do not give contradictory advice.**

Tasks:

- a) Identify the dialog acts required.**
- b) Identify the important entities that must be maintained.**
- c) Generate three possible responses satisfying all constraints.**
- d) Compare the responses based on:**
 - Coherence
 - Constraint satisfaction
 - Politeness
 - Relevance
- e) Select the best response and justify your answer.**
- f) Explain what happens if entity coherence is not maintained.**

Coding:

```
conversation = [
```

```
    "I submitted my assignment, but I am worried that I may have made many mistakes."
```

```
]
```

```
def identify_dialog_act(text):
```

```
    text = text.lower()
```

```
    if "worried" in text or "mistakes" in text:
```

```
        return ["Reassure", "Advise"]
```

```
return ["Inform"]
```

```
def identify_entities(text):
```

```
    entities = []
```

```
    if "assignment" in text.lower():
```

```
        entities.append("assignment")
```

```
    if "mistakes" in text.lower():
```

```
        entities.append("mistakes")
```

```
    if "i" in text.lower():
```

```
        entities.append("you")
```

```
    return entities
```

```
text = conversation[0]
```

```
acts = identify_dialog_act(text)
```

```
entities = identify_entities(text)
```

```
responses = [
```

```
    "Don't worry; it is normal to feel concerned about mistakes after submitting an assignment. You can review the feedback carefully and use it to improve your work, which will help you feel more confident next time.",
```

```
    "It is okay to be worried, but submitting your assignment is already an important achievement. Review any feedback you receive and use it to improve your understanding, so you can feel more confident about your future assignments.",
```

```
    "You do not need to worry too much about the mistakes because they can become opportunities to learn. Review your assignment feedback carefully and use it to improve your work and become more confident."
```

```
]
```

```
print("DIALOG ACTS")
```

```
print("-" * 40)
```

```
for act in acts:
```

```
    print(act)
```

```
print("\nIMPORTANT ENTITIES")
```

```

print("-" * 40)

for entity in entities:

    print(entity)

print("\nPOSSIBLE RESPONSES")

print("-" * 40)

for i, response in enumerate(responses, 1):

    print("\nResponse", i)

    print(response)

print("\nBEST RESPONSE")

print("-" * 40)

print(responses[0])

```

The screenshot shows a Python IDLE Shell window with the following output:

```

===== RESTART: C:/Users/ADMIN/OneDrive/ドキュメント/NLP/ass python/co5 at1 q2.py
=====
DIALOG ACTS
-----
Reassure
Advise

IMPORTANT ENTITIES
-----
assignment
mistakes
you

POSSIBLE RESPONSES
-----

Response 1
Don't worry; it is normal to feel concerned about mistakes after submitting an assignment. You
can review the feedback carefully and use it to improve your work, which will help you feel
more confident next time.

Response 2
It is okay to be worried, but submitting your assignment is already an important achievement.
Review any feedback you receive and use it to improve your understanding, so you can feel
more confident about your future assignments.

Response 3
You do not need to worry too much about the mistakes because they can become opportunitie
s to learn. Review your assignment feedback carefully and use it to improve your work and be
come more confident.

BEST RESPONSE

```

3. Word Sense Disambiguation and Meaning Representation

Source Sentence:

"Priya sat on the bank and watched the boats moving across the water."

Note:

The word "bank" can refer to:

- **A financial institution**
- **The side of a river**

Constraints:

- 1. Resolve the correct word sense using contextual information.**
- 2. Identify important entities and actions.**
- 3. Represent the meaning using predicate logic.**
- 4. Preserve semantic relationships.**
- 5. Maintain coherence in the generated paraphrase.**

Tasks:

- a) Identify the possible meanings of bank.**
- b) Select the correct sense and justify your decision using contextual constraints.**
- c) Write a predicate logic representation using suitable predicates such as:**

sit(x)

bank(y)

beside(x,y)

watch(x,z)

boat(z)

move(z)

water(w)
- d) Generate a simple English paraphrase without ambiguity.**
- e) Explain how Word Sense Disambiguation improves Machine Translation.**

Coding:

sentence = "Priya sat on the bank and watched the boats moving across the water."

possible_senses = [


```

    "Financial institution",

    "Side of a river"

]

context_words = [

    "sat",

    "boats",

    "water",

    "moving"

]

river_words = ["boats", "water", "river", "shore"]

Financial_words = ["money", "account", "cash", "loan"]

river_score = 0

financial_score = 0

for word in context_words:

    if word in river_words:

        river_score += 1

    if word in financial_words:

        financial_score += 1

if river_score > financial_score:

    selected_sense = "Side of a river"

else:

    selected_sense = "Financial institution"


print("WORD SENSE DISAMBIGUATION")

print("-" * 40)

```

```
print("Sentence:", sentence)
```

```
print("\nPossible meanings:")
```

```
for sense in possible_senses:
```

```
    print("-", sense)
```

```
print("\nSelected meaning:")
```

```
print(selected_sense)
```

```
print("\nPredicate Logic")
```

```
print("-" * 40)
```

```
print("sit(Priya, Bank)")
```

```
print("bank(Bank)")
```

```
print("beside(Priya, Bank)")
```

```
print("watch(Priya, Boat)")
```

```
print("boat(Boat)")
```

```
print("move(Boat)")
```

```
print("water(Water)")
```

```
print("across(Boat, Water)")
```

```
print("\nUnambiguous Paraphrase")
```

```
print("-" * 40)
```

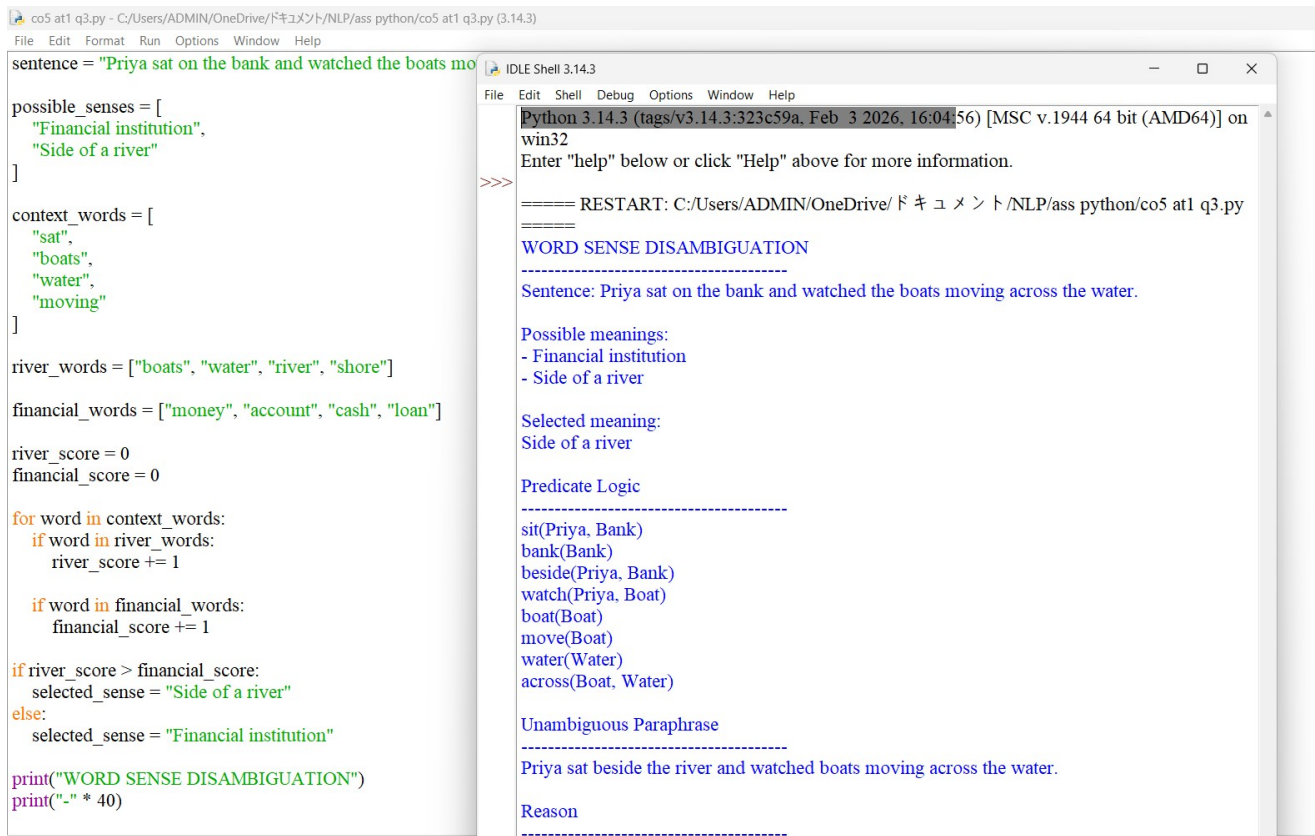
```
print("Priya sat beside the river and watched boats moving across the water.")
```

```
print("\nReason")
```

```
print("-" * 40)
```

```
print("The words boats and water indicate that bank means the side of a river.")
```

```
print("WSD helps machine translation choose the correct meaning of ambiguous words.")
```



The image shows a screenshot of a Python script and its execution output. The script is located in a file named 'co5 at1 q3.py' and is being run in an IDLE Shell window. The script performs Word Sense Disambiguation (WSD) on the sentence "Priya sat on the bank and watched the boats moving across the water." by comparing the scores of two possible meanings for the word "bank": "Financial institution" and "Side of a river".

```
sentence = "Priya sat on the bank and watched the boats moving across the water."

possible_senses = [
    "Financial institution",
    "Side of a river"
]

context_words = [
    "sat",
    "boats",
    "water",
    "moving"
]

river_words = ["boats", "water", "river", "shore"]
financial_words = ["money", "account", "cash", "loan"]

river_score = 0
financial_score = 0

for word in context_words:
    if word in river_words:
        river_score += 1

    if word in financial_words:
        financial_score += 1

if river_score > financial_score:
    selected_sense = "Side of a river"
else:
    selected_sense = "Financial institution"

print("WORD SENSE DISAMBIGUATION")
print("-" * 40)
```

The output of the script, displayed in the IDLE Shell window, is as follows:

```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

==== RESTART: C:/Users/ADMIN/OneDrive/ドキュメント/NLP/ass python/co5 at1 q3.py ====

WORD SENSE DISAMBIGUATION
-----
Sentence: Priya sat on the bank and watched the boats moving across the water.

Possible meanings:
- Financial institution
- Side of a river

Selected meaning:
Side of a river

Predicate Logic
-----
sit(Priya, Bank)
bank(Bank)
beside(Priya, Bank)
watch(Priya, Boat)
boat(Boat)
move(Boat)
water(Water)
across(Boat, Water)

Unambiguous Paraphrase
-----
Priya sat beside the river and watched boats moving across the water.

Reason
-----
```

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Q. No. / Criteria Level	Problem Understanding	Constraint Analysis	Solution Development	Application of Concepts	Justification	Sub. Total	Total %
1	7	7	6	6	6	32	93
2	7	7	6	6	6	32	
3	6	6	6	6	5	29	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____